

Segunda Prova de Linguagens de Programação
- DCC024 -
Sistemas de Informação

Nome: _____

“Eu dou minha palavra de honra que não trapacearei neste exame.”

Número de matrícula: _____

As regras do jogo:

- A prova é sem consulta.
- Quando terminar, não entregue nada além do caderno de provas para o instrutor.
- Quando escrever código, a sintaxe correta é importante.
- Cada estudante tem direito a fazer uma pergunta ao instrutor durante a prova. Traga o caderno de provas quando vier à mesa do instrutor. São perguntas: “Posso fazer uma pergunta?” ou “Quanto tempo falta?”
- A prova termina uma hora e quarenta minutos após seu início.
- Seja honesto e lembre-se: **você deu sua palavra de honra.**

Alguns conselhos:

- Escreva sempre algo nas questões, a fim de ganhar algum crédito parcial.
- Se não entender a questão, e já tiver gasto sua pergunta, escreva a sua interpretação da questão junto à resposta.
- A prova não é difícil, ela é divertida, então aproveite!

Tabela 1: Pontos acumulados (para uso do instrutor)

Questão 1	Questão 2	Questão 3	Extra

Questão Extra (0.5 Pontos): A Parábola do Filho Pródigo aparece no Evangelho de Lucas, 15:11–32. Escreva um sinônimo para a palavra “*Pródigo*”, usando um verbete de quatro a 16 caracteres.

1. Essa questão diz respeito ao programa abaixo, implementado em C++.

```
#include <iostream>
int sum(int a, int b) {
    std::cout << "Sum of ints\n";
    return a + b;
}
double sum(double a, double b) {
    std::cout << "Sum of doubles\n";
    return a + b;
}
int main() {
    std::cout << "The sum is " << sum(1, 2) << std::endl;
    std::cout << "The sum is " << sum(1.2, 2.1) << std::endl;
}
```

- (a) (2 Pontos) O programa acima ilustra um tipo de polimorfismo conhecido como sobrecarga, ou *overloading*. Essa forma de polimorfismo seria um tipo de polimorfismo *ad-hoc* (quantidade finita de tipos) ou universal *quantidade infinita de tipos*? Explique sua resposta
- (b) (3 Pontos) Não existe um análogo perfeito para polimorfismo de sobrecarga em Python. Por que?
- (c) (5 Pontos) Escreva um programa “equivalente” ao código acima, em Python. Note que C++ e Python são linguagens diferentes, então—evidentemente—uma equivalência perfeita não será possível.

2. O programa abaixo encontra “Triplas Pitagóricas”. Três números A , B e C formam uma tripla pitagórica caso a seguinte equação seja verdade: $C^2 = A^2 + B^2$:

```
def is_triple(a, b, c):
    return c*c == a*a + b*b

def solve_triple(n):
    for a in range(1, n): # Note que não incluímos o número 11
        for b in range(1, n):
            for c in range(1, n):
                if is_triple(a, b, c):
                    print a, b, c
```

- (a) (5 Pontos) Implemente, em Prolog, um predicado `is_triple(A, B, C)` que seja verdade quando os números `A`, `B` e `C` formarem uma tripla pitagórica, sendo `C` o valor da hipotenusa e `A` e `B` o valor dos catetos¹.

[illegible]

Escreva seu programa na grade acima, um caracter por célula. Você não precisa usar todos os espaços (tampouco todas as linhas: é possível implementar o predicado com menos que cinquenta caracteres!).

- (b) (5 Pontos) Implemente, em Prolog, um predicado `solve_triple(A, B, C)`, similar à função de mesmo nome implementada em Python logo acima. O predicado `solve_triple(A, B, C)` deve ser verdadeiro sempre que as seguintes condições forem verdadeiras:

- o predicado `is_triple(A, B, C)` é verdade.
- $A \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.
- $B \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.
- $C \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

Escreva seu programa na grade abaixo, um caracter por célula (Não precisa usar todos os espaços).

[illegible]

¹Não ouse perguntar o que é uma hipotenusa ou um cateto!

3. O programa abaixo está escrito na linguagem de programação D. Você não precisa conhecer essa linguagem para resolver a questão.

Programa escrito na linguagem D

```
01 int divBy(int n) {
02     return 1/n;
03 }
04
05 void greetThePlanet(int n) {
06     import std.stdio;
07     writeln("Hello, world!");
08 }
09
10 void main() {
11     greetThePlanet(divBy(0));
12 }
```

Saída do programa acima:

```
$> Floating point exception
(core dumped)
```

Programa escrito na linguagem D com avaliação preguiçosa:

```
01 int divBy(int n) {
02     return 1/n;
03 }
04
05 void greetThePlanet(lazy int n) {
06     import std.stdio;
07     writeln("Hello, world!");
08 }
09
10 void main() {
11     greetThePlanet(divBy(0));
12 }
```

Saída do programa acima:

```
$> "Hello, world!"
```

- (a) (4 Pontos) O primeiro programa causa uma exceção, enquanto o segundo programa termina sem erro. Explique qual a semântica da palavra chave “**lazy**” no segundo programa que causa essa diferença de comportamento.
- (b) (1 Pontos) Escreva o nome de uma linguagem de programação cujo comportamento padrão seja similar àquele visto na implementação da função **greetThePlanet** do programa da esquerda.
- (c) (1 Pontos) Escreva o nome de uma linguagem de programação cujo comportamento padrão seja similar àquele visto na implementação da função **greetThePlanet** do programa da direita.
- (d) (4 Pontos) “Efeito colateral” é o nome dado a qualquer ação executada por um programa que altera o estado do sistema ao qual aquele programa está inserido. Exemplos de efeitos colaterais são operações de entrada e saída de dados e operações de escrita em memória. A avaliação preguiçosa de expressões que causam efeitos colaterais poderia modificar a semântica de programas? Explique sua resposta.